

Cybersecurity Hackathon

Track H: Media & Civic Trust (H1)

- **Strengths:** Represents core cybersecurity principles: **Data Integrity, Non-Repudiation, and Chain of Custody.**
- **Cybersecurity Considerations:** Tackles forensic integrity, cryptographic hashing, and clear reporting for non-technical stakeholders (e.g., judges/magistrates).

Why Track H is the Winning Pick:

1. **Pure Infosec Core:** It touches the fundamental **Integrity** pillar of the CIA Triad. Unlike Track D (which is heavily Data Science/NLP-focused) or Track B (which leans heavily on UX/Identity adoption), Track H is pure security engineering.
2. **Deterministic & Bulletproof Demos:** AI models (Track D) can hallucinate or fail live. Cryptographic hashes do not lie. Demonstrating a file modification live and showing a Merkle root break makes an impression on judges.
3. **Full Compliance with Hackathon Rules:**
 - **Rule 05 (Honest Limits):** Easy to explain limits (e.g., SHA-256 collision resistance vs. disk I/O speed).
 - **Rule 06 (Offline First):** Evidence collection often happens in the field without internet; local offline hashing with background sync fits naturally.

Free Tools Arsenal for Track H

Using free and open-source tools:

1. Cryptography & Forensic Core

- **Python `hashlib` & `cryptography`:** For generating cryptographic hashes (SHA-256, BLAKE2) and digital signatures.
- **`pymerkle` / Custom Merkle Tree implementation:** To build tamper-evident hash chains and trees for evidence logs.
- **`ExifTool` / `pyexiftool`:** To extract and verify file metadata (timestamps, device information) without modifying source files.

2. Backend & Local Storage

- **FastAPI (Python):** Lightweight, fast framework to handle backend upload pipelines, hashing, and audit log APIs.
- **SQLite:** Local database for storing metadata, hash chains, and local chain-of-custody audit logs.

3. Frontend & Presentation UI

- **Streamlit or React + Tailwind CSS:** For building two distinct views:
 1. *Officer Field Interface:* Mobile-responsive upload, hash generation, offline queue indicator.
 2. *Judicial / Panel View:* Plain English verification report showing timestamp, chain of custody history, and a green/red "Integrity Validated" status.

4. Synthetic Data Generation

- **Faker (Python library):** To auto-generate realistic case details, officer names, and mock logs without using real personal data.

1-Week Timeline & Preparation Routine

Day 1: Architecture, Threat Modeling & Rule Alignment

- **Task:** Define threat models (e.g., Rogue officer modifies file on flash drive; transit corruption).
- **Cybersecurity Action:** Define the cryptographic hash algorithm (e.g., SHA-256) and chain-of-custody data schema.
- **Deliverable:** Architecture diagram detailing local storage, hashing pipeline, and synchronization flow.

Day 2: Build the Core Forensic Hashing Engine

- **Task:** Write the backend service that receives evidence files (videos, PDFs, images, logs).
- **Cybersecurity Action:** Process files in stream chunks (to handle files >100GB without running out of RAM) and generate SHA-256 checksums instantly.
- **Deliverable:** Working Python module that outputs file metadata, chunked hashes, and cryptographic signatures.

Day 3: Immutable Audit Log (Chain of Custody)

- **Task:** Build the append-only audit log tracking every user who accesses, views, or exports the evidence.
- **Cybersecurity Action:** Implement hash-chaining (where each log entry contains the hash of the previous log entry). If any entry is modified, the entire chain breaks.
- **Deliverable:** Tamper-evident ledger service storing historical access logs.

Day 4: Offline Capabilities & Syncing (Hackathon Rule 06)

- **Task:** Account for network power outages.
- **Cybersecurity Action:** Build a local queue mechanism (IndexedDB/SQLite) that saves evidence and signed hashes locally when offline, and automatically syncs with the central ledger when connected.

- **Deliverable:** Simulated offline-to-online sync demonstration.

Day 5: Non-Technical Judicial Dashboard

- **Task:** Create the interface for judges/panel members.
- **Cybersecurity Action:** Translate complex cryptographic verifications into a clear, jargon-free report.
- **Deliverable:** UI screen displaying: *"Status: Authentic"*, *"Collected by: Officer A"*, and a simple visual timeline of custody.

Day 6: Red Teaming, Edge Cases & "Failure Modes"

- **Task:** Deliberately attempt to break your own system.
- **Cybersecurity Action:**
 - Alter 1 byte in a video/image file and verify that the system flags it as tampered.
 - Edit a database log directly and show how hash-chain verification catches it.
- **Deliverable:** Documented list of limits and failure modes (per Hackathon Rule 05).

Day 7: Final Polish & Pitch Presentation

- **Task:** Practice the live presentation.
- **Cybersecurity Action:** Prepare a live demo:
 1. Upload evidence file (generates hash).
 2. Simulate network cut (shows offline functionality).
 3. Modify the file secretly live in hex/text editor.
 4. Run verification -> **RED WARNING: File Tampered!**
 5. Present plain-English report to "Judge".

Team Role Distribution

- **Backend Engineer:** Develops the Python API (FastAPI/Flask), manages the SQLite database, and builds the synchronization endpoints for offline data.
- **Frontend 1 (Field Ops):** Builds a mobile-responsive Progressive Web App (PWA) for officers to upload evidence and handles offline storage (IndexedDB/Service Workers) when networks fail.
- **Frontend 2 (Judicial UI):** Builds a clean desktop dashboard that translates complex cryptographic validation into plain English for non-technical judges.
- **Cybersecurity Engineer:** Designs the SHA-256 hash chaining architecture, generates synthetic, anonymized test evidence, and acts as the "Red Team" to deliberately tamper with files for the final demo.

Technical Implementation Details

- **The Hashing Pipeline:** When Frontend 1 uploads a file, the Cybersecurity script processes it in chunks (to prevent RAM overload on large files) using SHA-256. The

Backend stores this hash alongside the officer's ID and timestamp.

- **The Chain of Custody:** Every time the file is viewed, transferred, or downloaded, the Backend creates a new log entry. The Cybersecurity logic hashes this new entry together with the hash of the *previous* entry, creating an unbreakable chain.
- **Offline Tolerance:** Frontend 1 uses a Service Worker. If an officer has no signal, the app hashes the file locally on the device using a JavaScript crypto library, stores it in IndexedDB, and automatically POSTs it to the Backend once the connection returns.
- **The Demo Strategy:** During the pitch, the Cybersecurity engineer will secretly change one pixel of an image or one character in a text file. Frontend 2 will instantly display a red warning, proving tamper detection to the judges.

Frontend 1: Field Operations PWA (React)

- **Offline Storage & Hashing:** Use browser-native `window.crypto.subtle` for instant client-side SHA-256 calculation and the `idb` wrapper library to store pending uploads in IndexedDB when offline.
- **PWA Setup:** Configure the Vite PWA plugin (`vite-plugin-pwa`) or Workbox to ensure static assets load reliably without network connectivity.
- **Auto-Sync Listener:** Hook into `window.addEventListener('online', syncQueue)` to drain the IndexedDB queue and POST saved evidence payloads to the backend API as soon as the signal restores.

Frontend 2: Judicial Dashboard (React)

- **Plain-Language Reporting:** Translate raw backend JSON payloads (like timestamps, hashes, and chain logs) into clear visual cards using Tailwind CSS, keeping technical cryptographic mechanics behind simple status badges ("Authentic" vs "Tampered").
- **Interactive Custody Timeline:** Render an append-only visual progress bar mapping out every transfer or view event in chronological order.
- **Live Tamper Alert Component:** Set up state handlers that immediately render a red alert UI whenever a hash mismatch is detected during verification.

API Integration Strategy

- **CORS Management:** Have your Backend Engineer enable `CORSMiddleware` in FastAPI to allow cross-origin requests from the React development servers (`http://localhost:5173` or `http://localhost:3000`).
- **File Transfer:** Use `FormData` payloads over standard `axios` or native `fetch` multipart requests to transmit evidence files and client-side calculated hash metadata simultaneously to the backend.
- **Shared Mock Contracts:** The Cybersecurity Engineer provides mock JSON payload structures early on so both React engineers can build their UI state handlers independently from Day 1.

Neglect the distribution of roles in the team and the 7-days timeline to complete(follow it but we are going to do it in a day), I am going to complete the project on my own.

ICSC